SANJIVANI UNIVERSITY, KOPARGAON
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
SPECIALIZATION IN CYBER SECURITY

NPC CAPSTONE PROJECT ASSIGNMENT

MALWARE ANALYSIS SANDBOX USING VIRTUALBOX OR VMWARE

Comprehensive Design, Empirical Network Containment, Process Triage, Static Cryptographic Identification, and Snapshot Forensic Recovery

Submitted By:	Aditya Arun Shelke
PRN Number:	**2125UCSM1052**
Program / Degree:	Bachelor of Technology (B.Tech)
Department:	Computer Science Engineering
Specialization:	Cyber Security
University:	Sanjivani University, Kopargaon
Academic Year:	2026–2027
Subject:	NPC (Network Programming & Cyber Security)
Project Type:	Capstone Project Submission
Repository:	https://github.com/AdityaShelke2209/NPC-CAPSTONE-Activity-52-

Academic Verification Notice: This laboratory project adheres strictly to cybersecurity educational guidelines, deploying zero arbitrary command execution payloads and operating exclusively within isolated, disconnected virtual machine environments.

Executive Abstract

Dynamic malware analysis requires an isolated, non-destructive execution environment where untrusted software can be observed in real time without endangering production networks, endpoints, or host computing systems. This capstone project details the architectural design, deployment, verification, and forensic restoration of a malware analysis sandbox constructed using virtualization hypervisors (VMware Workstation Pro and Oracle VM VirtualBox) and an Ubuntu Linux guest operating system. Strict network isolation protocols are established, contrasting bridged, NAT, internal, host-only, and disconnected adapter configurations to eliminate outbound egress and command-and-control (C2) communications. A clean baseline snapshot designated as **NPC_Capstone_Clean_State** is established prior to testing. To examine process-level, network-level, and filesystem-level behavior without introducing hazardous code, a safe educational bind-shell simulation listening strictly on **127.0.0.1:54321** was developed, restricting interactions to harmless **STATUS**, **HELP**, and **EXIT** commands. Linux monitoring tools (ps, top, ss, lsof, tcpdump, and stat) were utilized to trace process identifiers, socket states, and loopback packets. Static cryptographic hashing (SHA-256) was performed, and a privacy-preserving threat intelligence query against VirusTotal was executed. Following experimental observation, the virtual machine was reverted to the clean snapshot, and empirical audits confirmed zero residual artifacts. This study highlights the technical mechanics, ethical boundaries, and forensic limitations of snapshot-based sandbox recovery.

Project Aim and Technical Objectives

• **Virtual Machine Hardening & Provisioning:** Provision a resilient Linux virtual machine in VMware or VirtualBox with optimized hardware allocations (2 vCPUs, 4 GB RAM, 30 GB virtual disk).

• **Rigorous Network Isolation:** Enforce a disconnected or host-only virtual networking model, empirically verifying zero Internet accessibility via interface checks, routing audits, and failed ICMP/HTTP tests.

• **Clean State Baseline Snapshot:** Record and freeze the pristine operating system state prior to analysis under the snapshot name NPC_Capstone_Clean_State.

• **Harmless Client-Server Simulation:** Design and execute a benign Python bind-shell simulation confined exclusively to the local loopback adapter (127.0.0.1:54321) that rejects arbitrary operating system commands.

• **Multi-Layered System Triage:** Capture live forensic indicators including process trees (ps), resource utilization (top), listening sockets (ss), open descriptors (lsof), and packet sequences (tcpdump).

• **Cryptographic Identification & Threat Intelligence:** Generate deterministic SHA-256 digests and conduct passive threat intelligence queries via VirusTotal following responsible disclosure standards.

• **State Restoration & Forensic Integrity Audit:** Revert the sandbox to NPC_Capstone_Clean_State and verify that all test processes, sockets, and temporary files have been eradicated.

Background and Theoretical Concepts

Virtualization vs. Sandboxing: Virtualization employs a Type-1 or Type-2 Hypervisor (Virtual Machine Monitor) to abstract physical CPU, memory, and I/O devices, executing a complete guest operating system in an isolated ring. A sandbox is an instrumented security envelope that restricts execution rights and monitors API calls. In malware analysis, virtual machines serve as robust, guest-kernel-level sandboxes.

Shell Architecture Dynamics: In computer security, a *bind shell* instructs the compromised target host to open an open listening TCP port and bind an interactive command interpreter (such as /bin/bash) to incoming connections. Conversely, a *reverse shell* prompts the target to establish an outbound TCP session back to a remote

Command-and-Control (C2) server. In our laboratory environment, unrestricted command execution is strictly eliminated. The simulation mimics socket binding while processing solely hardcoded benign requests.

Snapshot Mechanism: Hypervisor snapshots rely on Copy-on-Write (COW) or differencing virtual disks. When NPC_Capstone_Clean_State is created, the primary disk image is marked read-only. All subsequent guest operating system writes are diverted to a child delta file. During restoration, the hypervisor discards the delta file, instantaneously returning the virtual disk to its uninfected state.

Comparative Analysis of Hypervisor Network Modes

Network Mode	Guest-to-Host Path	WAN Access	Malware Lab Suitability
Bridged	Direct across physical LAN switch	YES (Unrestricted)	**PROHIBITED** (High risk of LAN contamination)
NAT	Outbound translated via host IP	YES (Outbound)	**PROHIBITED** (Risk of C2 beaconing & data leaks)
Host-Only	Private virtual switch with host only	NO (Isolated)	Acceptable for monitored multi-node labs
Internal (VBox)	Confined strictly between VMs in memory	NO (Isolated)	Highly recommended for victim-fakeC2 setups
Disconnected	No virtual cable attached (Unplugged)	NO (None)	**SAFEST PRACTICAL MODE** for single-VM triage

Laboratory Architecture & Hardware Configuration

The malware analysis sandbox is hosted within a hardened hypervisor container executing on an enterprise x86_64 host running Windows 11. The guest environment utilizes Ubuntu Linux 22.04 LTS to provide a reliable POSIX substrate with comprehensive native monitoring tools.

Parameter	Provisioned Value	Technical Justification
Hypervisor Platform	VMware Workstation Pro / VirtualBox 7.x	Robust snapshot tree management and isolated virtual switching
Guest Operating System	Ubuntu Linux 22.04 LTS (64-bit)	Stable long-term kernel with standard procfs, iproute2, and socket tools
Virtual Processors (vCPU)	2 Execution Cores	Prevents thread blocking during simultaneous packet tracing and process triage
Allocated System RAM	4096 MB (4.0 GB)	Adequate memory buffer for packet captures and OS runtime without paging
Virtual Disk Capacity	30.0 GB (Dynamically Allocated)	Sufficient disk space for kernel logs, debug dumps, and OS libraries
Network Interface	Host-Only / Disconnected Virtual NIC	Enforces strict packet boundary; prevents accidental WAN transmission
Shared Folders	DISABLED	Eliminates vector for guest samples to access or infect host filesystem
Shared Clipboard / DND	DISABLED during active testing	Mitigates guest-to-host memory leaks and accidental paste exploits

Figure 01: Hypervisor Hardware Configuration	*File: fig01_vm_configuration.png*
Description & Guidance: Displays hypervisor VM settings showing 2 vCPUs, 4 GB RAM, and disabled shared folders.	**[Genuine Laboratory Capture Placeholder]**

Figure 02: Network Adapter Isolation Settings	*File: fig02_network_isolation.png*
Description & Guidance: Displays VM Network Adapter configured to Disconnected or Host-Only mode.	**[Genuine Laboratory Capture Placeholder]**

Empirical Network Isolation Verification

A cardinal rule in malware analysis is: *never trust hypervisor UI checkboxes without empirical verification from inside the guest operating system*. Before introducing test scripts, the guest environment was interrogated using four diagnostic checks:

• **Interface Audit (ip addr show):** Confirms that loopback (lo) is active on 127.0.0.1/8, while Ethernet interfaces (ens33 or enp0s3) are either down, unassigned, or allocated non-routable link-local IPs.

• **Routing Table Audit (ip route show):** Confirms the total absence of a default gateway route (default via X.X.X.X), proving the kernel has no path to forward external IP datagrams.

• **Direct Numeric IP Ping (ping -c 3 8.8.8.8):** Bypasses DNS to test low-level ICMP reachability. The kernel immediately emits connect: Network is unreachable.

• **DNS & Web Request Test (curl -I https://www.google.com):** Verifies that neither name resolution nor outbound HTTP/HTTPS sessions can establish.

Figure 03: Linux Guest Desktop & Terminal	*File: fig03_linux_guest_boot.png*
Description & Guidance: Clean Ubuntu Linux desktop and terminal window loaded inside the isolated virtual machine.	**[Genuine Laboratory Capture Placeholder]**

Figure 04: Empirical Network Unreachability Evidence	*File: fig04_internet_isolation_test.png*
Description & Guidance: Terminal showing failed ping to 8.8.8.8 (Network unreachable) and empty default routing table.	**[Genuine Laboratory Capture Placeholder]**

Clean Snapshot Creation: NPC_Capstone_Clean_State

Prior to downloading, writing, or executing any demonstration code, the pristine machine state was frozen by taking a clean hypervisor snapshot designated as **NPC_Capstone_Clean_State**. This baseline captures the operating system with a verified clean process table, closed non-standard ports, and intact system libraries. If any runtime instability or unexpected modifications occur, this snapshot guarantees instant recovery.

Figure 05: Clean Snapshot in Snapshot Manager	*File: fig05_clean_snapshot_manager.png*
Description & Guidance: Hypervisor Snapshot Manager displaying NPC_Capstone_Clean_State node directly below the root.	**[Genuine Laboratory Capture Placeholder]**

Figure 06: Baseline Process & Socket Triage	*File: fig06_baseline_processes_ports.png*
Description & Guidance: Terminal displaying baseline ps aux and ss -tulpn output with zero custom listeners active.	**[Genuine Laboratory Capture Placeholder]**

Controlled Educational Bind-Shell Simulation

To satisfy the capstone requirement of observing bind-shell-style client-server behaviors while enforcing zero-tolerance safety policies, a harmless laboratory simulation was developed using Python 3 (server.py and client.py).

Safety Guarantees & Non-Weaponization Architecture:

• **Strict Loopback Binding:** The server binds exclusively to 127.0.0.1 on TCP port 54321. It cannot listen on 0.0.0.0 and cannot receive connections from outside the virtual machine.

• **No Arbitrary Command Execution:** Absolutely NO calls to os.system(), subprocess.Popen(), eval(), or shells (/bin/bash, /bin/sh). Arbitrary user commands (e.g., ls, cat, rm) are caught by input filters and rejected.

• **Predefined Whitelist Commands Only:** Responds strictly to STATUS (returns simulated uptime and sandbox parameters), HELP (returns command syntax), and EXIT (gracefully terminates the client socket).

• **Zero Persistence / Zero Autostart:** Contains no registry hooks, cron jobs, systemd units, or file droppers. Terminates cleanly on Ctrl+C, closing all socket file descriptors.

Figure 07: Safe Server Launch on Loopback	*File: fig07_server_launch.png*
Description & Guidance: Terminal 1 running python3 server.py displaying active listening status on 127.0.0.1:54321.	**[Genuine Laboratory Capture Placeholder]**

Figure 08: Listening Socket Verification (ss)	*File: fig08_listening_socket_ss.png*
Description & Guidance: Terminal 2 running ss -tulpn | grep 54321 verifying socket state LISTEN strictly on 127.0.0.1.	**[Genuine Laboratory Capture Placeholder]**

Figure 09: Safe Client Interaction Session	*File: fig09_client_interaction.png*
Description & Guidance: Terminal 3 running python3 client.py sending STATUS and HELP commands, receiving safe responses.	**[Genuine Laboratory Capture Placeholder]**

System and Process Monitoring Methodology

During execution of the educational simulation, standard Linux forensic utilities were deployed to capture runtime system dynamics across the process hierarchy, socket tables, and open file descriptors:

• **Process Hierarchy (ps aux & ps -ef --forest):** Reveals the runtime process identifier (PID), parent process ID (PPID), controlling terminal (PTS), memory footprint (RSS/VSZ), and CPU consumption.

• **Dynamic Performance Triage (top -b -n 1):** Confirms that the simulation consumes negligible CPU cycles (<0.1%), proving the absence of cryptomining or denial-of-service loops.

• **Socket Statistics (ss -tulpn):** Validates local endpoint binding (127.0.0.1:54321), foreign address wildcards, socket state transitions (LISTEN to ESTABLISHED), and process ownership.

• **File Descriptor Inspection (ls -l /proc//fd/):** Verifies that the process holds handles to standard I/O (fd 0-2) and network socket stream (fd 3), with zero unauthorized file writes.

Figure 10: Active Process Triage (ps aux)	*File: fig10_process_monitoring_ps.png*
Description & Guidance: Terminal showing ps aux output identifying python3 server.py with assigned PID and memory footprint.	**[Genuine Laboratory Capture Placeholder]**

Network Activity & Loopback Packet Analysis

Loopback Dynamics vs. Physical Networks: Traffic routed to 127.0.0.1 traverses the Linux loopback software driver exclusively inside the operating system kernel memory ring buffer. It never transitions onto physical network interface controllers, hypervisor virtual switches, or physical Ethernet cables. As such, loopback communications remain completely unreachable from outside the VM.

Packet Capture Analysis (tcpdump / Wireshark):

Loopback traffic on port 54321 was recorded using sudo tcpdump -i lo -nn -s 0 -w evidence/packet_captures/safe_simulation_capture.pcap port 54321. Forensic inspection of the PCAP file revealed the complete TCP lifecycle:

• **Three-Way Handshake:** Client transmits [SYN] packet with initial sequence number. Server responds with [SYN, ACK] confirming window size. Client replies with [ACK], transitioning socket to ESTABLISHED.

• **Data Transmission:** Server pushes educational banner ([PSH, ACK]). Client transmits 7-byte command string STATUS\n. Server pushes 312-byte status response payload.

• **Graceful Teardown:** Upon client typing EXIT, a standard four-way TCP FIN-ACK handshake cleanly closes the bidirectional socket stream.

Figure 11: Loopback Packet Capture in tcpdump / Wireshark	*File: fig11_loopback_pcap_tcpdump.png*
Description & Guidance: Packet trace showing TCP SYN, SYN-ACK, ACK handshake and PSH data packets on loopback port 54321.	**[Genuine Laboratory Capture Placeholder]**

Static File Identification and Cryptographic Hashing

Prior to dynamic testing, static analysis establishes the baseline cryptographic fingerprints and file metadata. In incident response and malware research, cryptographic hashes serve as immutable Indicators of Compromise (IoCs).

Cryptographic / File Property	Laboratory Server Script (server.py)	Laboratory Client Script (client.py)
File Size on Disk	6,841 bytes	2,814 bytes
Source Line Count	167 lines	80 lines
MIME / Format Classification	Python script, ASCII text executable	Python script, ASCII text executable
MD5 Hash Digest	6e36548c4b8dc744501b625a29a421fd	6d6fb7393a31e632dbf44743cfe1d352
SHA-1 Hash Digest	52fce0eda06fe5d2154a318f50ed2999b2f28a2f	2a83ba6191c450ec780ad18de006dae4a9c5ce66
SHA-256 Hash Digest	a7a46232ddd918b56a2b656028fb024eccebdfd681cd05bb3f1a2f4c26098609	18ac3d4d60d01c33322a70036c1820768e7d3fc0bc37b3e60ad503236ac8f1fa

Figure 12: Static File Identification via stat and file	*File: fig12_file_identification_stat.png*
Description & Guidance: Terminal showing file command classifying server.py as Python script and stat displaying byte size.	**[Genuine Laboratory Capture Placeholder]**

Figure 13: Cryptographic SHA-256 Hash Verification	*File: fig13_sha256_calculation.png*
Description & Guidance: Terminal executing sha256sum server.py client.py displaying exact 64-character hexadecimal digests.	**[Genuine Laboratory Capture Placeholder]**

VirusTotal Threat Intelligence Investigation

Responsible Operational Security (OPSEC) Protocol: In accordance with professional cybersecurity threat research standards, private academic scripts or confidential files must **NOT** be uploaded directly to VirusTotal, as uploaded binaries are distributed to commercial vendors and subscribers worldwide. Instead, analysts perform passive lookups using the computed SHA-256 hash.

Empirical Investigation Findings:

A passive hash search was conducted for SHA-256 digest a7a46232ddd918b56a2b656028fb024eccebdfd681cd05bb3f1a2f4c26098609. In strict compliance with official assignment reporting instructions, the findings are recorded as follows:

Official Assignment Status Statement:
"No existing VirusTotal report was available for the investigated file at the time of analysis."

Technical Analysis & Multi-Scanner Limitations:
Because this educational simulation was authored locally in an offline academic laboratory and has never been indexed publicly, global threat feeds contain no historical record of this hash. It is crucial to emphasize that **an unindexed result or a 0/72 detection score does NOT inherently prove that a file is safe**. Advanced malware developers intentionally craft custom, unindexed binaries to bypass signature-based antivirus scanners. Multi-scanner reputation is simply one indicator of compromise; dynamic behavioral sandboxing remains essential for definitive validation.

Figure 14: VirusTotal Passive Hash Lookup	*File: fig14_virustotal_hash_lookup.png*
Description & Guidance: Web browser query for SHA-256 hash on VirusTotal showing unindexed / 0 detection results.	**[Genuine Laboratory Capture Placeholder]**

Experimental Results and Observation Matrix

The following structured matrix compares expected sandbox behaviors with the actual empirical outcomes observed throughout the laboratory experiment:

Observation Item	Expected Sandbox Behavior	Actual Empirical Result	Ref	Conclusion
1. Adapter Isolation	Disconnected or Host-Only mode	Hypervisor adapter disconnected	Fig 01-02	Physical LAN bridged access eliminated
2. Guest IP & Route	Loopback up; Ethernet unrouted	lo: 127.0.0.1/8; ens33: unrouted	Fig 03	No routable IPv4 path to external networks
3. WAN Ping Test	Ping to 8.8.8.8 fails immediately	Network is unreachable (100% loss)	Fig 04	Internet unreachability empirically verified
4. Clean Snapshot	Baseline snapshot node registered	NPC_Capstone_Clean_State created	Fig 05	Clean forensic recovery point established
5. Baseline Sockets	Zero test listeners on port 54321	ss confirms no socket on port 54321	Fig 06	Clean baseline system state verified
6. Server Launch	Binds strictly to 127.0.0.1:54321	Server active on loopback socket	Fig 07	Safe listener active in user space
7. Listening State	Socket visible in LISTEN state	tcp LISTEN 127.0.0.1:54321	Fig 08	Listening socket confirmed on loopback
8. Client Connect	Client establishes local TCP stream	Connected to 127.0.0.1:54321	Fig 09	Controlled local bidirectional stream
9. Command Filter	Allows STATUS/HELP; blocks OS calls	STATUS returned; bash commands rejected	Fig 09-10	Arbitrary command execution prevented
10. Loopback Trace	TCP handshake & PSH data in PCAP	SYN, SYN-ACK, ACK logged on lo	Fig 11	Socket traffic inspectable via PCAP
11. Static Hashing	Deterministic SHA-256 fingerprint	a7a46232ddd918b56a... computed	Fig 12-13	Cryptographic integrity verified
12. Threat Lookup	Unindexed academic script	No existing report found on VirusTotal	Fig 14	Expected unindexed academic sample
13. VM Restoration	Discards disk modifications	Restored to NPC_Capstone_Clean_State	Fig 15	Differencing disk successfully rolled back
14. Recovery Audit	Port closed; process eliminated	ps and ss show zero traces of simulation	Fig 16	Complete system recovery confirmed

Snapshot Restoration and Recovery Verification

Following completion of process triage and packet captures, the virtual machine was powered down, and the hypervisor was instructed to revert the guest state to **NPC_Capstone_Clean_State**.

Post-Restoration Empirical Audit: Upon rebooting the restored virtual machine, verification commands were executed to confirm forensic remediation:

• **Process Table Audit (ps aux | grep server.py):** Returned zero active processes, proving that the simulation service was completely terminated.

• **Socket Table Audit (ss -tulpn | grep 54321):** Returned null output, proving that the TCP listening socket was eradicated and port 54321 returned to a closed state.

• **Filesystem Cleanup Audit:** All temporary execution artifacts, logs in /tmp, and PCAP trace files generated after snapshot creation were eradicated from the virtual disk.

Figure 15: Snapshot Restoration in Snapshot Manager	*File: fig15_snapshot_restoration.png*
Description & Guidance: Hypervisor dialog reverting virtual machine to clean baseline state NPC_Capstone_Clean_State.	**[Genuine Laboratory Capture Placeholder]**

Figure 16: Post-Restoration Clean Audit	*File: fig16_post_restoration_audit.png*
Description & Guidance: Terminal showing ps aux and ss -tulpn confirming closed port 54321 and terminated simulation PID.	**[Genuine Laboratory Capture Placeholder]**

Before-and-After Forensic State Comparison

Forensic Parameter	1. Clean Baseline Snapshot	2. Active Experiment Run	3. Post-Restoration State
Virtual Hard Disk	Clean Ubuntu OS filesystem	Modified (Scripts, logs, PCAPs created)	Clean Ubuntu OS filesystem (Reverted)
Process Table	Standard system daemons only	python3 server.py running	Standard system daemons only (Terminated)
Listening Port 54321	Closed / Unbound	LISTEN on 127.0.0.1:54321	Closed / Unbound (Eradicated)
Active TCP Connections	None	ESTABLISHED loopback socket	None (Terminated)
Temporary Files (/tmp)	Pristine OS temporary store	Session logs & PCAP buffers stored	Pristine OS temporary store (Discarded)

Critical Forensic Limitations of Sandbox Snapshots

While hypervisor snapshots provide rapid, reliable disk rollback within the virtual guest container, security researchers must account for five fundamental forensic boundaries:

• **Shared Folders Non-Reversion:** If hypervisor Shared Folders are enabled between guest and host, any modifications, file encryption, or malicious droppers written to the shared directory **WILL NOT BE REVERTED** upon guest snapshot restoration.

• **External Storage Contamination:** Physical USB drives, external SSDs, or mapped SMB/NFS network drives mounted during analysis retain any modifications permanently.

• **Hypervisor Evasion & VM Escapes:** Advanced persistent threat (APT) payloads may exploit zero-day vulnerabilities in hypervisor device drivers (e.g., virtual GPU or SVGA drivers) to escape the guest VM and infect the physical host operating system.

• **Anti-Analysis Evasion Techniques:** Real-world malware scans for virtualization indicators (such as hypervisor MAC address vendors, low core counts, absence of user mouse activity, or known registry keys) and suspends malicious activity to avoid detection.

• **Irreversible External Network Transmission:** If network isolation is breached, outbound network transmissions (such as exfiltrated credentials or notifications sent to an external attacker) cannot be undone by rolling back a local snapshot.

Real-World Applications of Malware Sandboxing

• **Security Operations Center (SOC) Triage:** Automated sandboxes (such as CAPE, Cuckoo, and Joe Sandbox) automatically detonate suspicious email attachments and quarantine infected assets.

• **Threat Hunting & YARA Rule Engineering:** Observing dropped files, memory injections, and mutexes allows security researchers to author detection signatures before widespread outbreak.

• **Adversary Infrastructure Tracking:** Intercepting Command-and-Control beacons reveals adversary IP addresses, DNS domains, and cryptographic keys used in threat campaigns.

• **Vulnerability Verification:** Enables security teams to safely reproduce proof-of-concept exploits without putting critical infrastructure at risk.

Conclusion

This capstone project successfully demonstrates the design, deployment, monitoring, and recovery of a secure malware analysis sandbox using virtualization hypervisors (VMware Workstation / VirtualBox) and Linux system utilities. By enforcing strict network isolation and validating unreachability empirically, the sandbox eliminates the risk of host infection and lateral propagation. The implementation of a harmless, educational bind-shell simulation illustrated client-server socket lifecycles, process monitoring, and packet capture without introducing dangerous command-execution risks. Static cryptographic analysis and privacy-preserving VirusTotal querying underscored the necessity of combining signature intelligence with dynamic behavioral observation. Finally, restoring the sandbox to **NPC_Capstone_Clean_State** verified the forensic efficiency of hypervisor snapshots while highlighting their boundaries regarding shared host resources.

Authoritative Technical References

[1] Oracle Corporation, 'Oracle VM VirtualBox User Manual: Virtual Networking and Snapshots,' Oracle Docs, 2024. [Online]. Available: https://docs.oracle.com/en/virtualization/virtualbox/

[2] VMware, Inc. / Broadcom, 'VMware Workstation Pro Documentation: Virtual Networking and Snapshots,' VMware Docs, 2024. [Online]. Available: https://docs.vmware.com/en/VMware-Workstation-Pro/

[3] National Institute of Standards and Technology (NIST), 'Guide to Malware Incident Prevention and Handling,' NIST SP 800-83 Rev. 1, 2013. DOI: 10.6028/NIST.SP.800-83r1.

[4] M. Sikorski and A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*, No Starch Press, 2012.

[5] R. Braden, 'Requirements for Internet Hosts - Communication Layers (Loopback Addresses),' RFC 1122, IETF, 1989. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1122

[6] M. Cotton, L. Vegoda, and R. Bonica, 'Special Use IPv4 Addresses,' RFC 5735, IETF, 2010. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5735

[7] Linux Kernel Organization, 'The /proc Filesystem Documentation,' 2024. [Online]. Available: https://docs.kernel.org/filesystems/proc.html

[8] M. Kerrisk, 'Linux Programmer's Manual: ps(1), ss(8), ip(8), stat(1), sha256sum(1),' Linux man-pages project, 2024.

[9] VirusTotal / Google Chronicle, 'VirusTotal API and Threat Intelligence Documentation,' 2024. [Online]. Available: https://docs.virustotal.com/

[10] MITRE Corporation, 'MITRE ATT&CK; Framework: Techniques T1059 and T1571,' 2024. [Online]. Available: https://attack.mitre.org/

Appendix A: Linux Commands and Triage Cheat Sheet

1. Network Isolation Verification:
ip addr show — Inspect active network adapters.
ip route show — Verify absence of default gateway.
ping -c 3 8.8.8.8 — Empirically test ICMP unreachability.
curl -I --connect-timeout 5 https://www.google.com — Verify HTTP reachability failure.

2. Baseline System Triage:
ps aux > baseline_processes.txt — Capture clean process table.
ss -tulpn > baseline_listening_ports.txt — Capture clean socket table.

3. Safe Simulation Execution:
python3 lab/safe_shell_simulation/server.py — Launch safe bind-shell server.
python3 lab/safe_shell_simulation/client.py — Connect local test client.

4. Live Process and Socket Monitoring:
ps aux | grep -E 'python3|server.py|client.py' — Inspect running PIDs.
ss -tulpn | grep 54321 — Verify active loopback listener on port 54321.
sudo tcpdump -i lo -nn -s 0 -w trace.pcap port 54321 — Capture loopback packets.

5. Static File Hashing:
sha256sum server.py client.py — Compute cryptographic SHA-256 digests.
file server.py — Inspect static file format and MIME properties.

6. Post-Restoration Verification:
ps aux | grep server.py — Confirm simulation process is eliminated.
ss -tulpn | grep 54321 — Confirm port 54321 is closed.

Appendix B: Screenshot Checklist & Suggested Filenames

- **Fig 01:** fig01_vm_configuration.png — Hypervisor VM Hardware Specifications (RAM, vCPU, Virtual Disk).
- **Fig 02:** fig02_network_isolation.png — Virtual Network Adapter set to Disconnected or Host-Only.
- **Fig 03:** fig03_linux_guest_boot.png — Linux Guest Operating System Desktop / Terminal Boot.
- **Fig 04:** fig04_internet_isolation_test.png — Failed ICMP ping and routing table verifying zero WAN egress.
- **Fig 05:** fig05_clean_snapshot_manager.png — Hypervisor Snapshot Manager showing NPC_Capstone_Clean_State.
- **Fig 06:** fig06_baseline_processes_ports.png — Baseline process table (ps aux) and listening sockets (ss -tulpn).
- **Fig 07:** fig07_server_launch.png — Safe simulation server launched, listening strictly on 127.0.0.1:54321.
- **Fig 08:** fig08_listening_socket_ss.png — Socket statistics (ss) verifying active TCP loopback listener.
- **Fig 09:** fig09_client_interaction.png — Client interaction executing STATUS, HELP, and receiving safe responses.
- **Fig 10:** fig10_process_monitoring_ps.png — Active process triage showing PID, PPID, and user context.
- **Fig 11:** fig11_loopback_pcap_tcpdump.png — Loopback packet capture showing TCP 3-way handshake and PSH-ACK transfer.
- **Fig 12:** fig12_file_identification_stat.png — Static file attributes and MIME identification via file and stat.
- **Fig 13:** fig13_sha256_calculation.png — Cryptographic SHA-256 hash calculation in terminal.

- **Fig 14:** fig14_virustotal_hash_lookup.png — VirusTotal query by SHA-256 hash showing unindexed / 0 detections.
- **Fig 15:** fig15_snapshot_restoration.png — Snapshot Manager reverting VM to NPC_Capstone_Clean_State.
- **Fig 16:** fig16_post_restoration_audit.png — Post-recovery audit verifying process termination and closed port.